Bridge Solver Online (BSOL) — Interface Guide

John Goacher / Dr. Klaus Krtschil
12 September 2026

1. Overview

Bridge Solver Online (BSOL) is a static browser application for inspecting, analysing, and playing bridge boards. It uses Bo Haglund's Double Dummy Solver (DDS), compiled to WebAssembly. Board analysis runs locally in the browser; a web server is only needed to deliver the HTML, JavaScript, WebAssembly, and data files.

The application is delivered as `index.html`. The source page is maintained as fragments in `html/` and assembled with `scripts/build-html.js`. BSOL can be opened directly, embedded in an iframe, or called with URL parameters.

2. Starting BSOL

2.1 Start without parameters

Open the application URL without a query string:

```
https://example.org/path-to-bsol/index.html
```

The start panel allows the user to:

- select a local `.pbn`, `.lin`, or `.dlm` file;
- drag and drop a supported file;
- paste board text into the page;
- retrieve boards from the clipboard, subject to browser permission;
- enter a board manually.

After loading a file, BSOL displays the board and, when Traveller/result data is available, the corresponding result views. Navigation, analysis, play, options, and help controls are available through the application menus.

2.2 Open a single board with parameters

A board can be supplied directly in the query string:

```
https://example.org/path-to-bsol/index.html?
board=14&dealer=E&vul=NS&north=...
```

Parameter names are case-insensitive. Values are validated. Invalid compulsory values prevent the board from being loaded; invalid optional values are ignored.

Compulsory board parameters

`board` is the board identifier and may contain up to 15 characters.

`dealer` is one of `N`, `S`, `E`, or `W`.

`vul` is one of `NS`, `EW`, `All`, or `None`.

`north`, `south`, `east`, and `west` contain the four hands. Suits are ordered Spades, Hearts, Diamonds, Clubs and separated by dots. The card `10` is written as `T`:

```
north=Q853.KJT82.K5.J5
west=KQJ..A9764.JT432
```

BSOL checks that all four hands are present, contain the expected cards, and do not contain duplicates.

Optional board parameters

Parameter	Description	Example
`dd`	20 hexadecimal Double Dummy trick values	`dd=33423444442222323a3`
`optimumscore`	Optimum score for the board	`optimumscore=4SN+2`
`contract`	Played contract	`contract=4SxN`
`declarer`	Declarer (`N`, `S`, `E`, or `W`)	`declarer=W`
`leadcard`	Opening lead, card followed by suit	`leadcard=QS`
`title`	Title shown above the board	`title=Championship`
`event` / `eventid`	Event name or identifier	`event=Championship`
`club`	Club name	`club=Bridgeclub`
`pair_number`	Pair number for a single board	`pair_number=402`
`direction`	Display direction for a single-board result	`direction=NS`
`compare`	Enables comparison data where supported	`compare=true`
`analyse`	Requests automatic analysis	`analyse=true`
`display`	Initial result view: `allpairs`, `personal`, or `board`	`display=board`
`lang`	Display language, currently `en` or `de`	`lang=en`
`nav`	Show or hide the top navigation (`0` hides it)	`nav=0`
`debug`	Enables diagnostic output	`debug=true`

2.3 Load a LIN board

The `lin` parameter accepts a BBO LIN string containing a board definition, auction, and/or play

record:

```
https://example.org/path-to-bsol/index.html?
lin=pn%7CNorth,East,South,West%7C...
```

URL-encode the value when necessary. BSOL can display the recorded auction and play sequence and can continue the play where the data permits.

2.4 Load a file by URL

The `file` parameter loads a remote `.pbn`, `.lin`, or `.dlm` file:

```
https://example.org/path-to-bsol/index.html?
file=https%3A%2F%2Fexample.net%2Fevent.pbn
```

The remote host must permit the request with an appropriate CORS header, for example `Access-Control-Allow-Origin`. An accompanying Traveller/result file may be supplied with `xml`:

```
?
file=https%3A%2F%2Fexample.net%2Fevent.pbn&xml=https%3A%2F%2Fexample.net%2Fresults.json
```

The XML/result URL must correspond to the loaded board file. The browser smoke test for the XML conversion is `test/xml-smoke.html`.

3. User interface

The page uses a fluid layout with a maximum content width of approximately 1000 pixels. Import controls wrap inside their fieldset when the available width is limited. Board and Traveller tables preserve their useful width and scroll within their local container on narrow screens; the page itself does not intentionally overflow.

The board/play area uses a softer Arial-based sans-serif font. Read-only legends and status areas use calmer surfaces, while action buttons use a stronger teal contrast. Dialogs, progress indicators, error messages, and help panels are constrained to the viewport on small screens.

The interface supports keyboard focus indicators, accessible names for controls, live updates for the board number and play position, and reduced-motion preferences.

4. Analysis and play

Click **Analyse** to calculate makeable contracts and optimum results. The DDS WebAssembly module is executed in one main Worker for play and in background Workers for parallel analysis. If a Worker or the WebAssembly runtime fails, BSOL stops the affected work and shows a localized error message instead of leaving the operation silently stuck.

Select a declarer and contract to inspect the play view. Available cards are presented as controls. The navigation buttons allow stepping through boards, results, and recorded play where the loaded data supports it.

5. Development and validation

The application uses classic global scripts; script order in `html/head.html` is significant. Edit the HTML fragments rather than the generated `index.html`.

Install dependencies and run the project checks:

```
npm ci
npm test
npm run build:html
npm run validate:html
```

The Node regression suite is in `test/regression.test.js`. The browser-only XML smoke test is `test/xml-smoke.html`. CI also checks JavaScript syntax and that the generated `index.html` is current.